

B.Sc. Information Technology — Semester VI

Internet of Things (IoT)

Course Code: 25UBIT603

UNIT 1

Introduction to IoT

Topics Covered:

- ① Definition, Characteristics & Applications of IoT
- ② IoT Reference Architecture (5-Layer & 3-Layer)
- ③ IoT Ecosystem & Enabling Technologies
- ④ Embedded Systems and IoT
- ⑤ Communication Models & Protocols (Request-Response, Pub-Sub)

IoT Exam: 27th March | Credits: 4 | Max Marks: 150

1. Definition, Characteristics & Applications of IoT

1.1 What is IoT?

The Internet of Things (IoT) refers to the vast network of physical devices, vehicles, appliances, and objects that are embedded with sensors, software, and network connectivity — enabling them to collect and exchange data over the Internet without human-to-human or human-to-computer interaction.

- Term coined by Kevin Ashton in 1999 at MIT's Auto-ID Center
- Core idea: Machines communicate autonomously — M2M (Machine-to-Machine)
- IoT bridges the physical and digital worlds

1.2 Traditional Internet vs IoT

Traditional Internet	Internet of Things (IoT)
Connects computers, phones, tablets	Connects physical objects and 'things' to network
Human initiates most interactions	Devices interact autonomously (M2M)
Data entered manually by humans	Data collected automatically via sensors
Limited to digital world	Bridges physical and digital worlds
Example: Browsing a website	Example: Smart fridge reordering milk automatically

1.3 Key Characteristics of IoT

#	Characteristic	Description
1	Connectivity	Devices must be connected to network (Wi-Fi, Bluetooth, Zigbee, Cellular) — foundational requirement
2	Identity	Every device has unique identity via IPv6 address, RFID tag, or unique device ID
3	Intelligence	Rule-based automation to ML-based predictions — enables smart decision-making
4	Dynamic Nature	System adapts to real-time changes — devices join/leave network automatically
5	Enormous Scale	15-20 billion devices in 2024, projected 75+ billion by 2030
6	Sensing	Sensors are the 'eyes and ears' — perceive and collect data from physical world
7	Heterogeneity	Devices vary in hardware, OS, protocols — must all work together
8	Safety & Security	Critical for hospitals, power grids, vehicles — data privacy and encryption essential
9	Interoperability	Devices from different manufacturers must communicate using standard protocols
10	Energy Efficiency	Battery-powered devices must operate for years — use LoRa, Zigbee, sleep modes

1.4 Applications of IoT

Domain	Application	Example
Smart Home	Automated lighting, security, energy management	Google Home, Amazon Alexa controlling appliances
Healthcare (IoMT)	Remote patient monitoring, wearables, smart hospitals	Fitbit, smartwatch ECG, connected insulin pumps
Smart Agriculture	Soil monitoring, irrigation control, crop management	Soil moisture sensor auto-triggers irrigation
Industrial IoT (IIoT)	Predictive maintenance, factory automation	Vibration sensor predicts machine failure
Smart Cities	Traffic management, waste management, smart lighting	Smart streetlights, smart parking sensors
Transportation	Vehicle tracking, fleet management, autonomous vehicles	GPS tracking, connected cars
Smart Energy	Smart meters, grid monitoring, energy optimization	Smart electricity meter sends usage data automatically

2. IoT Reference Architecture

2.1 5-Layer IoT Architecture

The 5-layer model is the most detailed and academically standard IoT architecture. Data flows upward from the physical world to the application layer.

Layer	Name	Role	Examples
Layer 5 (Top)	Application Layer	Delivers application-specific services to user — user-facing interface	Mobile apps, web dashboards, Google Home, SCADA
Layer 4	Processing / Middleware Layer	'Brain' of IoT — receives data, processes, stores, makes decisions	AWS IoT, Google Cloud IoT, Azure IoT Hub, ML engines
Layer 3	Network / Transport Layer	Transmits collected data from sensors to processing layer reliably	Internet gateways, routers, network hubs
Layer 2	Data Link / Gateway Layer	Bridge between physical devices and network — aggregates and pre-processes data	Raspberry Pi gateway, edge computing nodes
Layer 1 (Bottom)	Perception / Physical Layer	'Eyes, ears, hands' of IoT — senses environment and actuates actions	DHT11 sensor, PIR sensor, servo motor, GPS module

📌 *Data Flow: Physical Layer → Gateway Layer → Network Layer → Processing Layer → Application Layer*

2.2 3-Layer IoT Architecture

The 3-layer model is a simplified version — merges some layers for easier conceptual understanding. Widely used in academic textbooks.

Layer	Name	Maps to 5-Layer
Layer 3 (Top)	Application Layer	Same as Application Layer in 5-layer model
Layer 2 (Middle)	Network Layer	Combines Gateway Layer + Network/Transport Layer from 5-layer model
Layer 1 (Bottom)	Perception Layer	Same as Physical/Perception Layer in 5-layer model

2.3 5-Layer vs 3-Layer — Comparison

Aspect	5-Layer Model	3-Layer Model
Number of Layers	5 layers	3 layers
Detail Level	More detailed and comprehensive	Simplified and conceptual
Usage	Detailed system design and research	Academic introductions and textbooks

Aspect	5-Layer Model	3-Layer Model
Gateway Layer	Separate dedicated layer	Merged into Network Layer
Processing Layer	Separate dedicated cloud/middleware layer	Merged into Application Layer
Suitability	Complex enterprise-grade IoT deployments	Quick understanding and exam revision

3. IoT Ecosystem & Enabling Technologies

3.1 IoT Ecosystem — 6 Components

Component	Description	Examples
IoT Devices & Hardware	Physical 'things' — sensors, actuators, microcontrollers, SBCs, RFID	Arduino, ESP32, Raspberry Pi, DHT11 sensor
Connectivity & Communication	Networks and protocols for device communication	Wi-Fi, Bluetooth, Zigbee, LoRaWAN, 4G/5G
IoT Gateway	Bridge between IoT devices and cloud — aggregates and pre-processes data	Raspberry Pi gateway, industrial gateways
Cloud Platform & Data Processing	Receive, store and process massive IoT data	AWS IoT, Google Cloud IoT, Azure IoT Hub
Analytics & AI/ML	Extract actionable insights from IoT data	Apache Kafka, Hadoop, TensorFlow Lite
Applications & User Interface	End-user apps and dashboards	Mobile apps, web dashboards, Alexa, SCADA

3.2 Enabling Technologies

1. Wireless Sensor Networks (WSN):

A network of spatially distributed autonomous sensors that monitor physical conditions (temperature, sound, pressure) and pass data cooperatively to a main location.

- Each node has: sensor + microcontroller + radio transceiver + power source
- Topologies: Star, Mesh, or Tree
- Protocols: Zigbee, Bluetooth LE, Z-Wave, 6LoWPAN
- Applications: Environmental monitoring, precision agriculture, healthcare

2. RFID (Radio Frequency Identification):

Uses electromagnetic fields to automatically identify and track tags attached to objects.

RFID Component	Description	Example
RFID Tag	Small chip + antenna attached to object — stores unique ID	Tag on warehouse product

RFID Component	Description	Example
RFID Reader	Emits radio waves, receives signals from RFID tags	Retail store checkout scanner
Passive Tags	No battery — powered by reader's radio waves — range ~1-10m	Library book tags, retail price tags
Active Tags	Has own battery — longer range ~100m — more expensive	Vehicle tracking, hospital asset monitoring

3. Cloud Computing in IoT:

Cloud Model	Role in IoT	Example
IaaS (Infrastructure as a Service)	Virtual machines and storage for IoT backend servers	AWS EC2 hosting IoT data ingestion service
PaaS (Platform as a Service)	Pre-built platforms for IoT device management and analytics	AWS IoT Core, Google Cloud IoT Core
SaaS (Software as a Service)	Ready-to-use IoT applications	Salesforce IoT Cloud
FaaS (Functions as a Service)	Serverless event-driven functions triggered by IoT events	AWS Lambda processes sensor alert → sends SMS

4. Big Data & Analytics:

- Volume — Billions of devices generating continuous data streams
- Velocity — Real-time data must be processed immediately
- Variety — Structured (sensor readings), semi-structured (JSON), unstructured (images)
- Technologies: Apache Hadoop (batch), Apache Kafka (streaming), Apache Spark (both)

5. AI/ML in IoT (AIoT):

- Anomaly Detection — ML detects unusual patterns (e.g., gas leak from sensor readings)
 - Predictive Maintenance — Predict equipment failure using vibration and temperature trends
 - Computer Vision — AI cameras detect defects in manufacturing, face recognition
 - Edge AI — ML models run directly on IoT devices (TensorFlow Lite) for low-latency
-

4. Embedded Systems and IoT

4.1 What is an Embedded System?

An embedded system is a computer system — a combination of processor, memory, and I/O peripherals — designed for a specific dedicated function within a larger system. Unlike general-purpose computers, they are purpose-built.

- Real-time operation — responds to events within strict time limits
- Limited resources — minimal memory and CPU power
- Power-constrained — must run on very little energy
- Single-purpose — designed for one specific task

4.2 Traditional Embedded vs IoT-Enabled Embedded

Traditional Embedded System	IoT-Enabled Embedded System
Standalone — no network connection	Connected to Internet — data sent to/from cloud
Fixed, pre-programmed behavior	Can receive OTA (Over-The-Air) firmware updates
Local decision making only	Cloud-powered decisions — remote monitoring and control
Example: Washing machine control board	Example: Smart washing machine controlled via phone app

4.3 Key IoT Hardware Platforms

Feature	Arduino Uno	ESP32	Raspberry Pi 4
Type	Microcontroller	Microcontroller (SoC)	Single-Board Computer (SBC)
Processor	8-bit AVR, 16 MHz	Dual-core 32-bit, 240 MHz	Quad-core 64-bit, 1.5 GHz
RAM	2 KB	520 KB	1-8 GB
Wi-Fi	No (needs shield)	Built-in 802.11 b/g/n	Built-in 802.11ac
Bluetooth	No	BLE + Classic	Bluetooth 5.0
Operating System	None (bare metal)	None / RTOS	Linux (Raspberry Pi OS)
Programming	C/C++ (Arduino IDE)	C/C++, MicroPython	Python, Node.js, C/C++
Cost	~\$5-25	~\$5-15	~\$35-80
Best Use Case	Sensor interfacing, motors	IoT Wi-Fi sensor nodes	Gateway, edge AI, media

5. Communication Models & Protocols

5.1 Request-Response Model (Client-Server)

In the Request-Response model, a client sends a request to a server and waits for a response. Always client-initiated and synchronous.

How it Works:

Step	Process
1	Client (IoT device/app) sends HTTP/CoAP request to server
2	Server receives and processes the request (reads sensor data)
3	Server sends back response with data and status code
4	Client receives response and processes it (displays/triggers action)

Characteristic	Request-Response Details
Communication Style	Synchronous — client waits for response before proceeding
Initiation	Always client-initiated — server is passive
State	Stateless (HTTP) — each request is independent
Coupling	Tightly coupled — client and server must be available simultaneously
Scalability	Less scalable for massive IoT deployments
Primary Protocol	HTTP/HTTPS (most common), CoAP (constrained IoT devices)
Use Cases	Querying sensor data, controlling actuators, REST APIs

CoAP — Request-Response for Constrained IoT Devices:

- Lightweight — 4-byte header vs HTTP's ~500-byte overhead — perfect for microcontrollers
- UDP-based — uses UDP instead of TCP — lower overhead
- RESTful — uses same GET/POST/PUT/DELETE methods as HTTP
- Designed specifically for IoT devices with limited resources

5.2 Publish-Subscribe Model (Pub/Sub)

In the Publish-Subscribe model, devices (publishers) send messages to a central Broker without knowing who receives them. Subscribers register interest in topics and receive messages automatically. Fully asynchronous.

Key Actors:

Actor	Role
Publisher	IoT device that generates and sends messages — does NOT know who subscribers are
Broker	Central server — receives all messages and routes them to appropriate subscribers

Actor	Role
Subscriber	Device/app that registers interest in a 'topic' and receives matching messages
Topic	Channel/category to classify messages — e.g., 'home/bedroom/temperature'

Characteristic	Publish-Subscribe Details
Communication Style	Asynchronous — publisher doesn't wait for subscribers
Coupling	Loosely coupled — publishers and subscribers don't know each other
Scalability	Highly scalable — one message reaches thousands of subscribers simultaneously
Reliability	QoS levels (0, 1, 2) control delivery guarantees
Primary Protocol	MQTT (most widely used IoT pub/sub protocol)
Use Cases	Real-time sensor data, event notifications, smart home automation

5.3 MQTT — The Dominant IoT Protocol

MQTT (Message Queuing Telemetry Transport) is a lightweight, binary messaging protocol designed specifically for IoT and M2M communication. Developed by IBM in 1999. Runs over TCP/IP.

MQTT QoS Levels — Very Important for Exams:

QoS Level	Name	Description	Use Case
QoS 0	At most once	Fire and forget — no acknowledgment — fastest but no guarantee	Non-critical sensor readings
QoS 1	At least once	Guaranteed delivery but may duplicate — acknowledged by receiver	Important events where duplicates acceptable
QoS 2	Exactly once	Guaranteed delivery without duplication — 4-way handshake — slowest but most reliable	Critical transactions, billing data

MQTT Topic Wildcards:

- Single-level wildcard (+) — matches exactly ONE topic level
 - Example: 'home+/temperature' matches 'home/bedroom/temperature' and 'home/kitchen/temperature'
- Multi-level wildcard (#) — matches ANY number of topic levels — must be last character
 - Example: 'home/#' matches ALL topics starting with 'home/'

5.4 Request-Response vs Publish-Subscribe — Comparison

Feature	Request-Response	Publish-Subscribe
Communication Style	Synchronous	Asynchronous

Feature	Request-Response	Publish-Subscribe
Coupling	Tightly coupled — client-server know each other	Loosely coupled — broker decouples them
Initiation	Client initiates every request	Publisher sends whenever data is available
Scalability	Lower — one-to-one communication	Higher — one-to-many or many-to-many
Protocol	HTTP, CoAP	MQTT, AMQP, WebSocket
Real-Time Updates	Client must poll repeatedly	Server-push — data delivered immediately
Use Case	REST APIs, device control commands	Sensor telemetry, real-time monitoring, alerts
Example	HTTP GET /temperature	MQTT PUBLISH home/temperature 24.5

5.5 Wireless Communication Technologies

Technology	Range	Data Rate	Power	Best For
Wi-Fi (802.11)	~100m	Up to 1 Gbps	High	Smart home, high-bandwidth IoT
Bluetooth LE (BLE)	~50m	1-2 Mbps	Very Low	Wearables, health monitors, beacons
Zigbee	~10-100m	250 Kbps	Very Low	Smart home mesh networks, building automation
Z-Wave	~30m	100 Kbps	Very Low	Smart home devices, home automation
LoRaWAN	~10 km	0.3-50 Kbps	Ultra Low	Smart agriculture, smart city, long-range sensors
NB-IoT	~15 km	60-250 Kbps	Very Low	Smart meters, asset tracking, remote monitoring
4G/5G Cellular	Wide area	Up to 20 Gbps (5G)	High	Connected vehicles, industrial IoT, mobile IoT

Unit 1 — Quick Revision Summary

Topic	Key Points
IoT Definition	Network of physical devices with sensors/connectivity to exchange data without human intervention Coined by Kevin Ashton 1999
IoT Characteristics	Connectivity Identity (IPv6) Intelligence Dynamic Nature Scale Sensing Heterogeneity Security Interoperability Energy Efficiency
IoT Applications	Smart Home Healthcare (IoMT) Agriculture IIoT Smart Cities Transportation Smart Energy
5-Layer Architecture	Physical → Gateway → Network → Processing (Cloud) → Application (Top)
3-Layer Architecture	Perception → Network → Application (simplified — 3 layers only)
Physical Layer	Sensors + actuators — 'eyes, ears, hands' of IoT
Gateway Layer	Bridge between sensors and Internet — protocol conversion + edge processing
Processing Layer	Cloud/middleware — data storage, analytics, ML — 'the brain'
Application Layer	User-facing apps, dashboards, voice interfaces
WSN	Wireless Sensor Networks — distributed sensors communicating wirelessly
RFID	Radio Frequency ID Passive tags (no battery, ~10m) Active tags (battery, ~100m)
Arduino	Microcontroller No built-in Wi-Fi Best for sensor/actuator interfacing
ESP32	Microcontroller Built-in Wi-Fi + BLE Most popular IoT platform
Raspberry Pi	Single-Board Computer Linux OS Best for gateway/edge computing
Request-Response	Client-initiated Synchronous Tightly coupled HTTP/CoAP One-to-one
Publish-Subscribe	Async Loosely coupled Broker-mediated MQTT One-to-many
MQTT QoS	QoS 0 = At most once QoS 1 = At least once QoS 2 = Exactly once
MQTT Wildcards	+ = single level wildcard # = multi-level wildcard (must be last)

Important Exam Questions

- What is IoT? Explain its definition and key characteristics (7 marks)
- Explain 5-Layer IoT Reference Architecture with all layers (7 marks)
- Compare 5-Layer and 3-Layer IoT Architecture (5 marks)
- Explain IoT Ecosystem with its 6 components (5 marks)
- What is RFID? Explain its components — passive vs active tags (5 marks)
- Compare Arduino, ESP32 and Raspberry Pi for IoT (5 marks)
- Explain Request-Response communication model with HTTP and CoAP (5 marks)
- Explain Publish-Subscribe model with MQTT protocol and QoS levels (7 marks)
- Compare Request-Response vs Publish-Subscribe with all features (5 marks)